

Contents

1	Introduction	1
2	Matching Problems and Algorithms	2
2.1	Definitions	2
2.2	Maximum Bipartite Matching	3
2.3	Gale–Shapley Algorithm	7
2.4	Applications and Further Matching Models	7
3	Polyhedral Approaches to Matching Problems	8

List of Figures

List of Tables

Chapter 1

Introduction

Chapter 2

Matching Problems and Algorithms

2.1 Definitions

Let $G = (V, E)$ be an undirected graph with $|V| = n$ vertices and $|E| = m$ edges. We call G bipartite if we can partition its vertex set into two sets V_1, V_2 such that for each edge $u, v \in E$, $u \in V_1$ and $v \in V_2$. If every vertex of V_1 is adjacent to every vertex of V_2 and $|V_1| = n_1, |V_2| = n_2$, then we call G a complete bipartite graph and denote it by K_{n_1, n_2} . By a path, we will always mean a simple path, i.e., a path in which no vertex is repeated. The length of a path is the number of edges it contains.

From set theory, we recall that the symmetric difference of two sets A and B , denoted by $A \triangle B$, is the set of elements that belong to A or B but not both. Thus, $A \triangle B = (A \setminus B) \cup (B \setminus A) = (A \cup B) \setminus (A \cap B)$. Also, if we want to add an element x to set A , we will use the shorthand $A + x := A \cup \{x\}$, and if we want to remove it $A - x := A \setminus \{x\}$.

We write $T(n) = O(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \leq cf(n)$ for all $n \geq n_0$. Most of the time, $T(n)$ will denote a bound on the worst-case running time of an algorithm defined on the positive integers. Similarly, we say that $T(n) = \Omega(f(n))$ if and only if there exist positive constants c and n_0 such that $T(n) \geq cf(n)$ for all $n \geq n_0$. Finally, we say that $T(n) = \Theta(f(n))$ if and only if $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$.

A matching of a graph $G = (V, E)$ is a subset of E such that each vertex is incident to at most one edge of the subset. The cardinality of a matching is the number of edges it contains. A vertex is called matched if it is incident to an edge of M . Otherwise, it is called unmatched or free. A matching is called maximal if no edge can be added without violating the matching property. A matching is maximum if no other matching has more edges. Clearly, every maximum matching is maximal (the converse is not always true). The following two definitions are fundamental in the study of matchings.

Definition 2.1.1. A path P in G with respect to a matching M is called alternating if its edges are alternatingly out of and in M .

Definition 2.1.2. A path P in G with respect to a matching M is called augmenting if it is alternating and its endpoints are free.

It is easy to see that every augmenting path has odd length. We will use the term M -augmenting whenever we want to emphasize the matching M .

2.2 Maximum Bipartite Matching

In this section, we present two famous algorithms for finding a maximum bipartite matching and prove their correctness.

The first algorithm is a direct application of the following theorem proved by Berge in 1957 [1] (although it was already observed by Petersen in 1891 [2]).

Theorem 2.2.1. *Let $G = (V, E)$ be a graph and let M be a matching in G . Then M is maximum if and only if there is no M -augmenting path in G .*

Proof. Suppose that M is maximum. For the sake of contradiction, assume that there exists an M -augmenting path P with u and v as its endpoints in G . Then we can see that the matching $M' = M \triangle E(P)$ is a valid matching in G and satisfies $|M'| = |M| + 1$. Indeed, if we remove all edges of P that belong to M and add the remaining ones to M , we add one more edge to M in total (because P has odd length). The new matching is valid because u and v were free, and they were not incident to any matching edge, and every internal vertex of P remains incident to exactly one edge. All vertices outside P are unaffected. As a result, the new matching M' is valid and its cardinality is greater than that of M . This is a contradiction.

Now suppose that G does not contain any M -augmenting path and that a matching M' exists in G such that $|M'| > |M|$. Consider the graph $M_0 = M \triangle M'$. Each vertex in M_0 may be matched in both matchings or in either one. So, every vertex has degree at most 2. Hence, every component of M_0 is either a path or a cycle of even length (otherwise, there would exist a vertex that is incident to two edges in the same matching). Because M_0 has more edges of M' than of M , this also means that there exists a path P with more edges of M' than of M , and because the edges are alternating, P is an M -augmenting path, which contradicts our assumption that G does not contain any M -augmenting path. $\square$

Augmenting-Path Algorithm. Based on the theorem above, we can write an algorithm that will try to find an augmenting path, alternate its edges (this will increase the size of the current matching by one), and repeat the whole process until we can't find any augmenting path. $N(v)$ represents all the vertices u such that $\{v, u\} \in E(G)$.

Algorithm 2.1 Augmenting Path Algorithm

Require: Graph $G = (V, E)$, vertex v

Ensure: Array `matched` that corresponds to the edges of the maximum matching

```

1: function DfsAugmentingPath(int  $v$ , bool visited[], int matched[])
2:   if visited[ $v$ ] then
3:     return False;
4:   end if
5:   visited[ $v$ ]  $\leftarrow$  True;
6:   for all  $u$  in  $N(v)$  do
7:     if matched[ $u$ ] =  $\emptyset$  or DfsAugmentingPath(matched[ $u$ ], visited, matched) then
8:       matched[ $u$ ]  $\leftarrow v$ ;
9:       return True;
10:    end if
11:  end for
12:  return False;
13: end function

```

Algorithm 2.1 is getting called from all vertices in the left part of bipartite graph (let's call it V_1). We will use an extra array called `visited` to track the vertices from V_1 that we have already visited across the path and we have to reset it to be false for all these vertices before the initial call. Array `matched` marks the matched vertices only from the right part of the bipartite graph (let's call it V_2). To get the edges of the matching, we simply take all the edges $\{v, \text{matched}[v]\}$ such that $\text{matched}[v]$ is not empty and store them to the matching set.

Algorithm 2.2 Maximum Bipartite Matching in $O(nm + n^2)$

Require: Graph $G = (V, E)$, vertex sets $V_1, V_2 \subseteq V$

Ensure: Array `matched` that corresponds to the edges of the maximum matching

```

1: function MainAugmentingPath(int  $v$ , bool visited[])
2:   for all  $v \in V_2$  do
3:      $\text{matched}[v] \leftarrow \emptyset$ ;
4:   end for
5:   for all  $v$  in  $V_1$  do
6:     for all  $u$  in  $V_1$  do
7:        $\text{visited}[u] \leftarrow \text{False}$ ;
8:     end for
9:     DfsAugmentingPath( $v$ , visited, matched);
10:  end for
11:   $\text{matching} \leftarrow \emptyset$ ;
12:  for all  $v$  in  $V_2$  do
13:    if  $\text{matched}[v] \neq \emptyset$  then
14:       $\text{matching} \leftarrow \text{matching} \cup \{\{\text{matched}[v], v\}\}$ ;
15:    end if
16:  end for
17: end function

```

The idea of the algorithm is that if the initial call is from a free vertex, say v , and we cannot find another free vertex u such that the edge $\{v, u\}$ exists, then we simply try to find a path (v, x, y) such that $\{v, x\}$ is not in the matching, but $\{x, y\}$ is, so we can start trying again from the vertex y , until we find a free vertex and then recursively alternate the path.

Lemma 2.2.2. *If we call `DfsAugmentingPath` from a free vertex v and augmenting path with endpoint v exists, the algorithm 2.1 will return `True` and alternate its edges.*

Proof. Suppose that a free vertex $u \in V_2$ exists such that $\{v, u\} \in E(G)$. Then from line 7 the algorithm successfully finds an augmenting path. Now suppose that such a vertex doesn't exist and $P = (v, v_1, v_2, \dots, u)$ any augmenting path with endpoint the vertex v . Since P is alternating, the edge $\{v, v_1\}$ does not belong to the matching, while $\{v_1, v_2\}$ does. Because the algorithm checks all neighbors of v (line 6), it will eventually examine v_1 (assuming that all the other neighbors - if they exist - return false), and line 7 does a recursive call for $\text{matched}[v_1] = v_2$ (note that v_2 is impossible to be visited, otherwise an alternative path $(v, \dots, v_2)$ exists and the algorithm would have already found another augmenting path from that recursive call to u or to another free vertex). Repeating the same argument along P , the algorithm eventually reaches the free vertex $u \in V_2$, where line 7 is true since $\text{matched}[u] = \emptyset$. The recursive calls then return `True` and update the array `matched` accordingly, thus alternating the edges of P . $\square$

From this, we get the following remark.

Remark 2.2.3. *After an initial call of `DfsAugmentingPath` from a free vertex $v \in V_1$, the only vertices that can become matched are v and one free vertex of V_2 . If no augmenting path is found, then the matching remains unchanged.*

Then, we can prove the following lemma.

Lemma 2.2.4. *Every initial call to `DfsAugmentingPath` takes a free vertex as parameter.*

Proof. This is obviously true in the first iteration. Assume that we are at iteration i , and that the statement holds for all previous iterations. Let v be the vertex from which we are about to initially call `DfsAugmentingPath`. If v is free, then we are done. If it is not, v must have become matched during some previous iteration. However, by Remark 2.2.3, in each iteration the only vertex of V_1 that can become matched is the vertex from which `DfsAugmentingPath` was initially called. Since `DFS AUGMENTING PATH` has never been initially called from v before iteration i , the vertex v could not have become matched earlier. $\square$

The output of the algorithm (set *matching*) is always a valid matching. Since the algorithm initializes with an empty matching and only updates it by alternating edges along valid augmenting paths starting from free vertices (Lemma 2.2.4), the maintained set of edges is always a valid matching. Now we have to prove that it is maximum.

Theorem 2.2.5. *Algorithm 2.2 returns a maximum matching.*

Proof. Let $v_1, v_2, \dots, v_k$ be the vertices of V_1 in the order in which Algorithm 2.2 processes them. For each $i \in \{0, 1, \dots, k\}$, let M_i be the matching after the first i iterations of the algorithm, with $M_0 = \emptyset$. Also, let G_i be the subgraph of G induced by the vertex set

$$\{v_1, v_2, \dots, v_i\} \cup V_2.$$

We will prove by induction on i that M_i is a maximum matching of G_i .

For $i = 0$, the graph G_0 has no vertices in V_1 , so the empty matching M_0 is trivially maximum.

Now assume that M_i is a maximum matching of G_i , and consider iteration $i + 1$, where the algorithm processes the vertex v_{i+1} . By Lemma 2.2.4, the initial call to `DfsAugmentingPath` is made from a free vertex. We consider two cases.

Case 1: `DfsAugmentingPath`(v_{i+1}) returns True. Then, by Lemma 2.2.2, the algorithm finds an augmenting path in G_{i+1} with endpoint v_{i+1} and alternates its edges. Therefore,

$$|M_{i+1}| = |M_i| + 1.$$

On the other hand, any matching of G_{i+1} can use at most one edge incident to the new vertex v_{i+1} . Hence, by removing that edge if it exists, we obtain a matching of G_i , and therefore every matching of G_{i+1} has size at most $|M_i| + 1$. Since M_i is maximum in G_i , it follows that M_{i+1} is a maximum matching of G_{i+1} .

Case 2: `DfsAugmentingPath`(v_{i+1}) returns False. Then, again by Lemma 2.2.2, there is no M_i -augmenting path in G_{i+1} with endpoint v_{i+1} . Since M_i is maximum in G_i , Theorem 2.2.1 implies that there is no M_i -augmenting path contained entirely in G_i . Thus, there is no M_i -augmenting path in G_{i+1} at all, because any augmenting path in G_{i+1} that is not

contained in G_i must involve the only new vertex of V_1 , namely v_{i+1} . Therefore, by Theorem 2.2.1, M_i is already a maximum matching of G_{i+1} . Since the matching does not change in this case, we have

$$M_{i+1} = M_i,$$

and so M_{i+1} is a maximum matching of G_{i+1} .

In both cases, M_{i+1} is a maximum matching of G_{i+1} . This completes the induction. $\square$

Before each search, initializing the visited array takes $O(n)$ time. We run Algorithm 2.1 for each vertex $v \in V_1$ searching for an augmenting path. To find an augmenting path, we will visit each vertex and each edge at most once, so the total complexity is $O(nm + n^2)$.

Hopcroft-Karp Algorithm.

2.3 Gale–Shapley Algorithm

2.4 Applications and Further Matching Models

Chapter 3

Polyhedral Approaches to Matching Problems

Bibliography

- [1] Claude Berge. Two theorems in graph theory. *Proceedings of the National Academy of Sciences*, 43(9):842–844, 1957.
- [2] Julius Petersen. Die theorie der regulären graphs, 1891.